

Reviewer Pack — Minimal Order of Geometric Invariant Theory Bounds Resolution...

Entry 0f4086585adb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 01:07:09 UTC

Minimal Order of Geometric Invariant Theory Bounds Resolution Proof Size

Entry ID: 0f4086585adb **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 01:07:09 UTC

1. Conjecture

Field A (mathematical branch): Geometric Invariant Theory **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

For every instance of a SAT problem with n variables, if the minimal order of geometric invariants for its associated projective variety is at least k , then the size of a resolution proof for that instance is upper-bounded by $O(k^3 \log(n))$.

Rationale (proposer's reasoning):

Geometric Invariant Theory (GIT) provides tools to study algebraic varieties and their symmetries. By applying GIT invariants to SAT instances, we may expose underlying geometric properties that influence the complexity of resolution proofs. This conjecture posits a direct connection between geometric complexity and proof size, suggesting that high-dimensional structures could lead to complex refutations.

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1b2abaa60b582297

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 90% of randomly generated 3-CNF formulas with n variables have resolution proof sizes bounded by $O(k^3 \log(n))$, where k is the minimal order of geometric invariants for their associated projective varieties, and falsified if any seed produces a proof size exceeding $O(k^3 \log(n))$ for a given k .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.80	SAFE	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'geometric invariant theory' AND 'resolution proof complexity' AND 'SAT problem' - 'projective variety' AND 'minimal order of invariants' AND 'resolution proof size' - resolution proof complexity IN 'geometric invariant theory'

Top relevant hits considered: - [s2:10.4230/LIPIcs.CCC.2020.12] Search problems in algebraic complexity, GCT, and hardness of generators for invariant rings - [s2:1911.03990] Implementing geometric complexity theory: on the separation of orbit closures via symmetries

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(clause[i] != -clause[j] for i in range(n) for j in range(i + 1, n)):
                clauses.append(clause)
        return clauses

    def resolution_size(clauses):
        # Simplified resolution algorithm to estimate size
        resolvents = set()
        while True:
            new_resolvents = set()
            for clause1 in clauses:
                for clause2 in clauses:
                    common_vars = [var for var in clause1 if -var in clause2]
                    if len(common_vars) == 1:
                        literal = common_vars[0]
                        resolvent = sorted(set(clause1 + clause2) - {literal, -literal})
                        if resolvent not in resolvents:
                            new_resolvents.add(tuple(resolvent))
            if not new_resolvents:
                break
            clauses.extend(new_resolvents)
```

```

        resolvents.update(new_resolvents)
    return len(clauses)

n = random.randint(5, 40)
clauses = generate_3cnf(n)
resolution_size_value = resolution_size(clauses)

# Minimal order of geometric invariants (simplified heuristic)
k = len(set(abs(lit) for lit in sum(clauses, [])))

if k == 0:
    return {
        "metric_name": "resolution_size",
        "metric_value": resolution_size_value,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

upper_bound = k**3 * math.log(n)

return {
    "metric_name": "resolution_size",
    "metric_value": resolution_size_value,
    "instances_tested": 1,
    "conjecture_holds": resolution_size_value <= upper_bound,
    "counterexample": "" if resolution_size_value <= upper_bound else f"upper_bound={upper_bound}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 7 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    total_metric_value = sum(result["metric_value"] for result in results)
    mean_metric_value = total_metric_value / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.9:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {'metric_name': 'resolution_size', 'metric_value': 66, 'instances_tested': 1, 'conjecture_hol
TRIAL: {'metric_name': 'resolution_size', 'metric_value': 46, 'instances_tested': 1, 'conjecture_hol
TRIAL: {'metric_name': 'resolution_size', 'metric_value': 20, 'instances_tested': 1, 'conjecture_hol
TRIAL: {'metric_name': 'resolution_size', 'metric_value': 36, 'instances_tested': 1, 'conjecture_hol
TRIAL: {'metric_name': 'resolution_size', 'metric_value': 50, 'instances_tested': 1, 'conjecture_hol
```

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a5d6330b.py", line 80, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a5d6330b.py", line 50, in run_trial
    resolution_size_value = resolution_size(clauses)
                           ^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a5d6330b.py", line 40, in resolution_si
    if resolvent not in resolvents:
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

TypeError: unhashable type: 'list'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents a definitive evaluation of the conjecture. | next: Investigate and fix the crash in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12442
2	propose	ollama_remote	glm4:latest	0	0	10656
3	preregistration	ollama_remote	glm4:latest	0	0	5890
4	novelty	ollama_remote	glm4:latest	0	0	4871
5	novelty	ollama_remote	glm4:latest	0	0	6491
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21234
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7959
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8241
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9460
10	judge	ollama_remote	glm4:latest	0	0	10922

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 98167 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/0f4086585adb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0f4086585adb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0f4086585adb.tar.gz` (if generated)